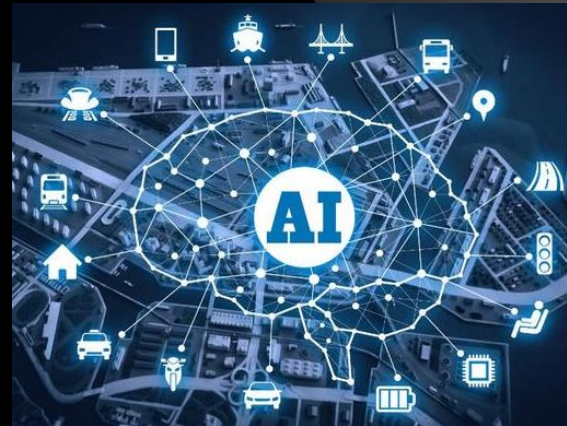


이미지 분류를 위한 인공지능 CNN 모델 원리와 구현



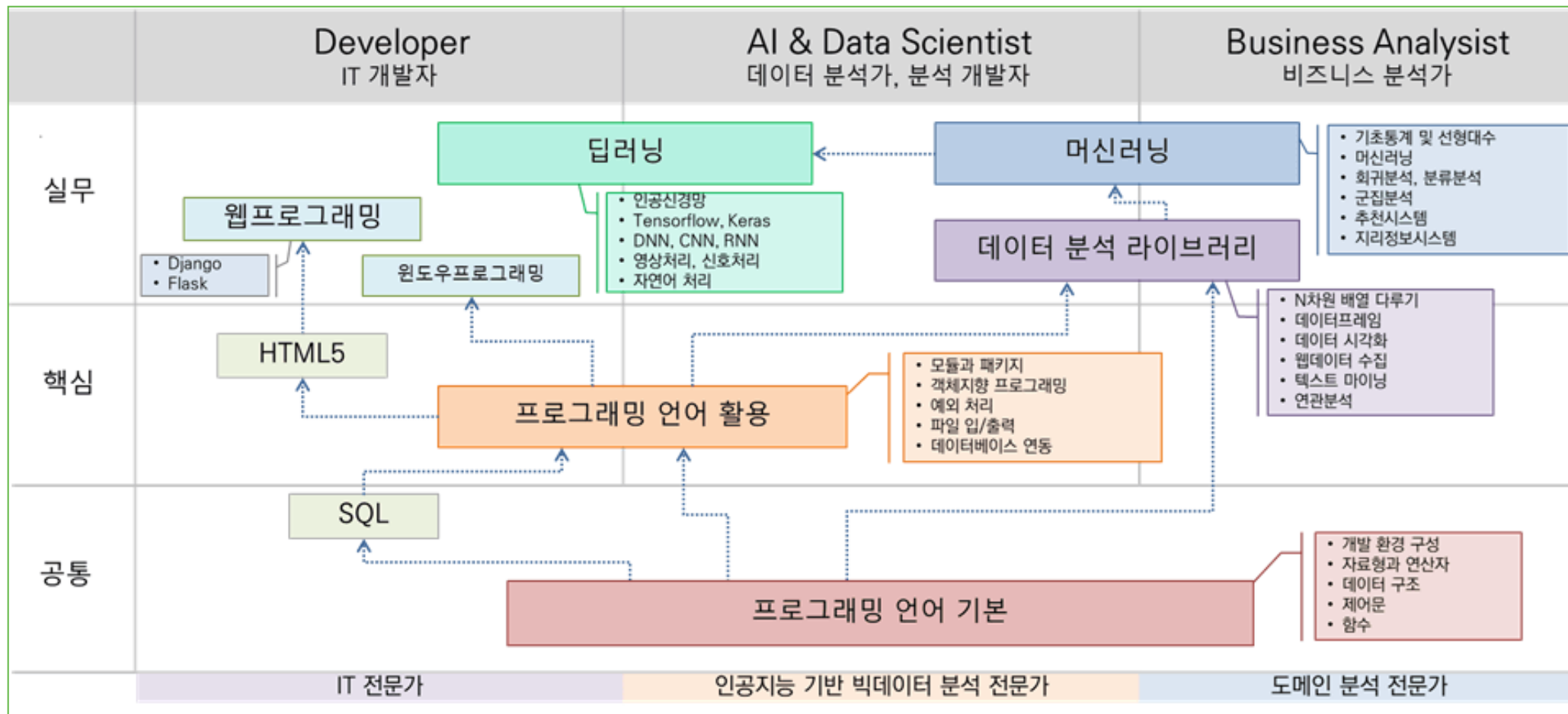
이 슬라이드에서 사용한 서체 :

- Open Sans(<https://ko.cooltext.com/Download-Font-Open+Sans>)
- KoPubWorld돋움체(<http://www.kopus.org/biz/electronic/font.aspx>)



인공지능 로드맵

이미지 분류를 위한 인공지능 CNN 모델 원리와 구현



실습 환경 (선택 1)

이미지 분류를 위한 인공지능 CNN 모델 원리와 구현

1. <https://repo.anaconda.com/archive/>에서 아나콘다 다운로드 및 설치
 - 강의장 운영체제는 Windows 10, GPU 2080
 - GPU를 사용하려면 [Anaconda3-2023.03-1-Windows-x86_64.exe](#) 파일을 내려받아 설치하세요.
 - 파이썬 개발 인터프리터, 주피터노트북 개발환경, 머신러닝 및 딥러닝을 위한 패키지 등이 설치됨
2. 아나콘다 설치 후 Anaconda prompt를 **관리자 권한으로 실행**하고 패키지 설치
 - 윈도우 10에서 GPU 사용: `pip install tensorflow==2.10`
 - 윈도우 11: `pip install tensorflow` 또는 `pip install tensorflow-gpu`

실습 환경 (선택 2)

이미지 분류를 위한 인공지능 CNN 모델 원리와 구현

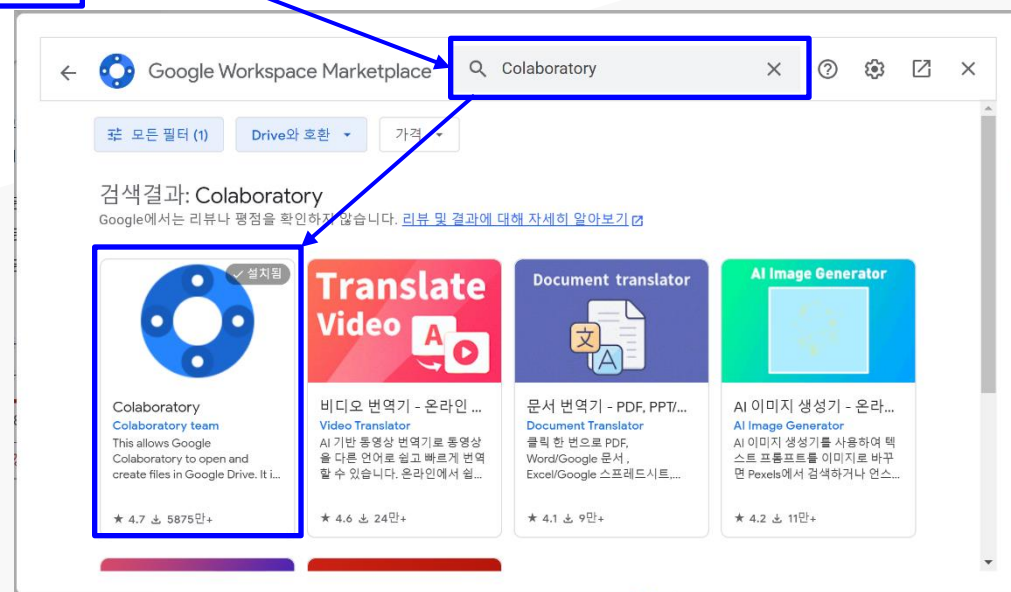
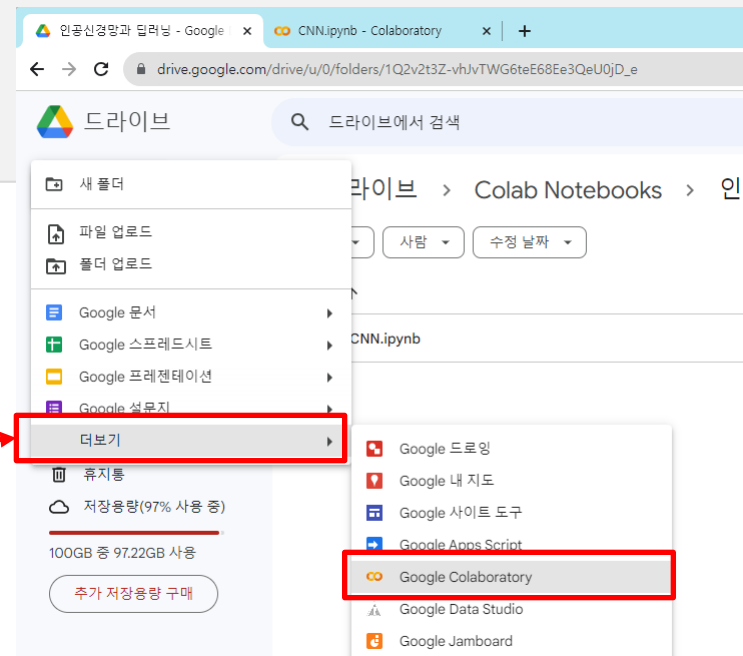
1. 개발 및 실행 환경은 Google Colab notebook을 사용합니다.

1. Chrome 브라우저에서 구글 계정으로 로그인
2. 구글 드라이브로 이동(<https://drive.google.com/>)

3. + 신규 -> 더보기 -> Google Colaboratory 선택
 - Colaboratory가 없을 경우 '연결할 앱 더보기'를 선택하고 Colaboratory를 검색하여 설치하세요.

4. 코드 작성 후 실행은 '컨트롤 + 엔터'

```
# 코랩에서 한글 그래프 적용하려면(세션 다시 시작)
!sudo apt-get install -y fonts-nanum
!sudo fc-cache -fv
!rm ~/.cache/matplotlib -rf
```



코드 작성 및 실행

이미지 분류를 위한 인공지능 CNN 모델 원리와 구현

colab.research.google.com/drive/1Reft5AUhyah9r_iC5sZsC1f0nvRk8_W

누구나 할 수 있는 인공지능 모델 구현.ipynb

파일 수정 보기 삽입 런타임 도구 도움말 오전 12:14에 마지막으로 저장됨

파일

- sample_data

+ 코드 + 텍스트

1 # 은 주석입니다.
2 # 코드를 작성하는 이 곳을 셀(Cell)이라고 부릅니다.
3 # 코드를 실행하려면 셀의 왼쪽에 있는 셀 실행 버튼을 클릭하거나 Shift+Enter 키를 누르세요.
4 # Shift+Enter로 실행하면 아래에 셀이 없으면 새로운 셀을 추가하고 포커스를 아래 셀로 이동합니다.
5 # Ctrl+Enter로 실행하면 현재 셀을 실행하고 포커스를 현재 셀에 머무르게 합니다.
6 # Alt+Enter로 실행하면 현재 셀을 실행하고 항상 아래에 새로운 셀을 만듭니다.

1 # 셀의 위쪽 또는 아래쪽 선 중간에 마우스를 올리면 '+ 코드'와 '+ 텍스트' 버튼이 보입니다.
2 # 새로운 코드 셀을 추가하려면 '+ 코드' 버튼을 클릭하세요.

+ 코드 + 텍스트

인공신경망 딥러닝 모델 구현

```
[ ] 1 from sklearn import datasets
    2 iris = datasets.load_iris()
    3 iris

{'data': array([[5.1, 3.5, 1.4, 0.2],
                [4.9, 3. , 1.4, 0.2],
                [4.7, 3.2, 1.3, 0.2],
                [4.6, 3.1, 1.5, 0.2],
                [5. , 3.6, 1.4, 0.2],
                [5.4, 3.9, 1.7, 0.4],
```

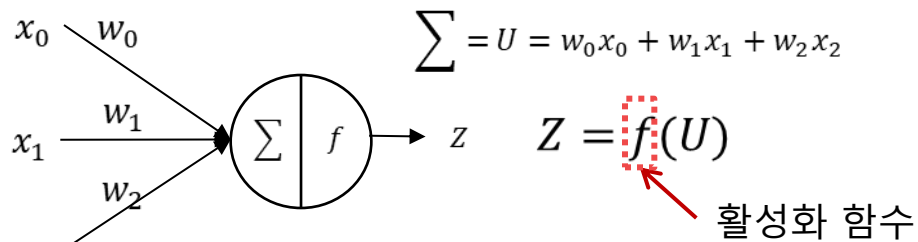
디스크 81.53 GB 사용 가능

Python 3 Google Compute Engine 백엔드에 연결됨

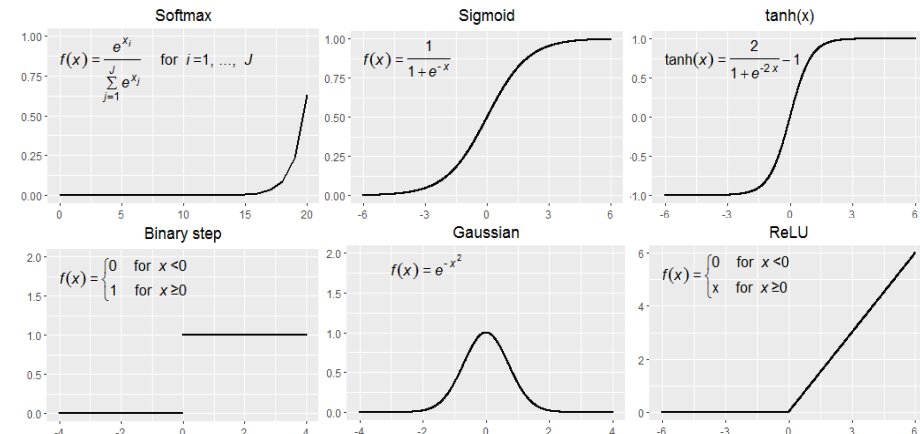
- Shift+Enter로 실행하면 아래에 셀이 없으면 새로운 셀을 추가하고 포커스를 아래 셀로 이동합니다.
- Ctrl+Enter로 실행하면 현재 셀을 실행하고 포커스를 현재 셀에 머무르게 합니다.
- Alt+Enter로 실행하면 현재 셀을 실행하고 항상 아래에 새로운 셀을 만듭니다.

- 셀의 위쪽 또는 아래쪽 선 중간에 마우스를 올리면 '+ 코드'와 '+ 텍스트' 버튼이 보입니다.
- 새로운 코드 셀을 추가하려면 '+ 코드' 버튼을 클릭하세요.

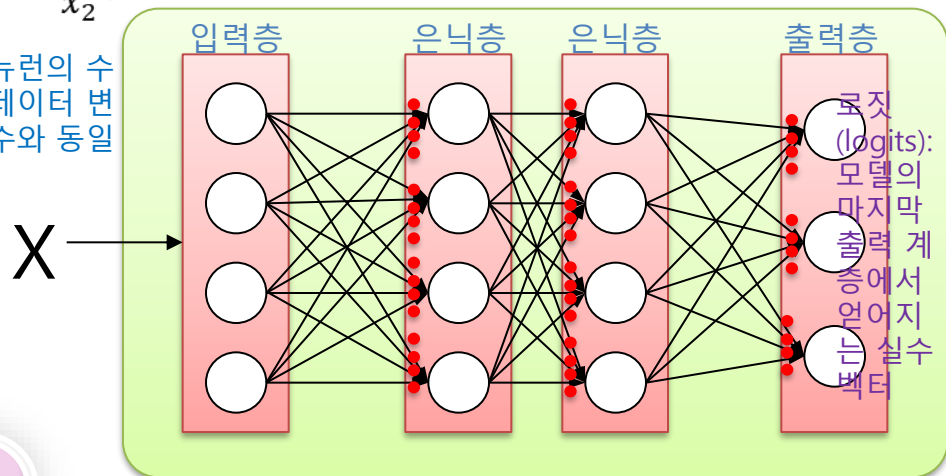
- #은 주석입니다.
- 코드를 작성하는 이 곳을 셀(Cell)이라고 부릅니다.
- 코드를 실행하려면 셀의 왼쪽에 있는 셀 실행 버튼을 클릭하거나 Shift+Enter 키를 누르세요.



소프트맥스 함수: 로짓을
입력으로 받아 각 클래스
에 대한 확률을 계산
하는 함수



입력층 뉴런의 수
는 입력데이터 변
수의 개수와 동일

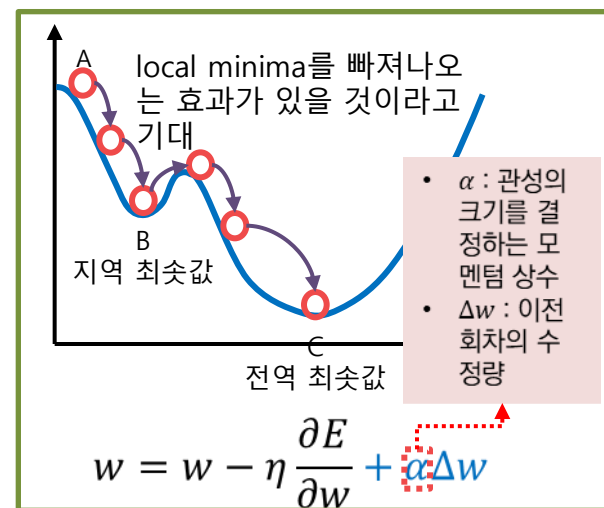
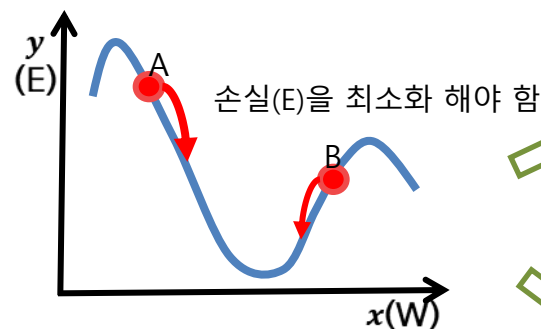


출력층 뉴런의 수는 분류
분석의 경우 분류 레이블
의 수이며, 회귀분석의
경우 1개

$\hat{y} \approx y$

차이(Loss, Error) { MAE, MSE, Crossentropy } 분류

손실함수



연구자가 고려해야 할 사항

- ▶ 은닉층의 수
- ▶ 은닉층별 뉴런의 수
- ▶ 활성화 함수
- ▶ 가중치 초기값
- ▶ 드롭아웃 비율
- ▶ 배치정규화
- ▶ 손실함수
- ▶ 옵티마이저, 학습률
- ▶ 학습 횟수(epochs)
- ▶ 배치 크기

가중치 업데이트

Gradient Descent

- weight
- 네트워크의 파라미터

$$w = w - \eta * \frac{\partial E}{\partial w}$$

- w에 대한 E의 편미분
- gradient
- 어떤 방향으로 학습할지

- Eta(learning rate)
- 한번에 얼마나 학습할지

- 편미분(偏微分, partial derivative)을 의미
- 기호는 델타의 변형
- 편-디(partial dee), 파셜디라고 부름

ChatGPT-3의 파라미터 개수는? 175B
ChatGPT-4의 파라미터 개수는? 1.7T

$$h = h + \left(\frac{\partial E}{\partial w} \right)^2$$

$$w = w - \eta \frac{1}{\sqrt{h}} \frac{\partial E}{\partial w}$$

2장. 텐서플로우(케라스)를 이용한 딥러닝 구현

인공신경망 딥러닝 알고리즘 구현과 실시간 객체 탐지

1 절. 텐서플로우 케라스

2장. 텐서플로우 케라스를 이용한 딥러닝 구현



케라스와 파이토치 비교

케라스와 파이토치 비교



Keras

- 높은 수준의 API를 제공함
- 코드가 직관적이고 사용하기 쉬움
- TensorFlow의 고수준 API로 통합되었음
- TensorFlow 생태계를 잘 활용할 수 있음



PyTorch

- 낮은 수준의 API를 제공
- 더 유연하고 커스터마이징하기 좋음
- 연구 커뮤니티에서 강력한 지지를 받고 있음
- 연구 결과를 빠르게 구현할 수 있는 라이브러리

기능	케라스	파이토치
모델 정의	Sequential 또는 Functional API 사용	nn.Module 클래스 상속
레이어 추가	model.add 또는 리스트 내 정의	__init__ 메서드 내에 레이어 정의
순전파	자동으로 처리	forward 메서드 내에서 직접 정의
모델 컴파일	model.compile 사용	없음, 직접 손으로 해야 함
모델 훈련	model.fit 사용	훈련 루프 직접 작성
손실 함수	loss 이자크 지정	nn.CrossEntropyLoss 등 손실 함수를

텐서플로우 케라스

1절. 텐서플로우 케라스



텐서플로우 케라스

- ▶ 텐서플로우의 케라스 API는 딥러닝 모델을 구축하고 훈련하기 위한 고수준 API로, 다양한 신경망 구조를 쉽게 구축할 수 있도록 설계되었음
- ▶ 텐서플로우 2.x이후부터 공식적으로 케라스를 포함함

- **tf.keras.Sequential**: 순차 모델은 여러 층을 연이어 쌓아서 신경망을 구축하는 데 사용됨. 각 층은 이전 층의 출력을 입력으로 받음
- **tf.keras.Model**: 사용자 정의 모델을 만들 때 이 클래스를 상속하여 모델을 정의할 수 있습니다. 이를 사용하면 복잡한 모델 구조를 구현할 수 있음.
- **tf.keras.layers**: 이 모듈은 다양한 종류의 층을 포함하고 있으며, 이를 사용하여 모델의 구조를 정의함. 예를 들어, **Dense** 층은 완전 연결된 층을 생성하고, **Conv2D** 층은 합성곱 신경망을 생성함
- **tf.keras.losses**: 손실 함수를 정의하는 데 사용. 모델의 훈련 중에 사용되며, 훈련 중에 최소화하고자 하는 목표 함수를 정의함. 예를 들어, **categorical_crossentropy**는 다중 클래스 분류 문제의 크로스 엔트로피 손실 함수임
- **tf.keras.optimizers**: 최적화 알고리즘을 선택하고 모델을 훈련하기 위해 사용. 예를 들어, **Adam**, **SGD**, **RMSprop**와 같은 최적화 알고리즘을 제공함.
- **tf.keras.metrics**: 모델의 성능을 평가하기 위한 메트릭을 정의하는 데 사용. 예를 들어, 정확도(**accuracy**)나 정밀도(**precision**)를 계산할 수 있음
- **tf.keras.callbacks**: 모델 훈련 중에 호출되는 콜백 함수를 정의하는 데 사용. 예를 들어, 학습률을 동적으로 조정하거나 모델 저장을 자동화하는 데 유용함

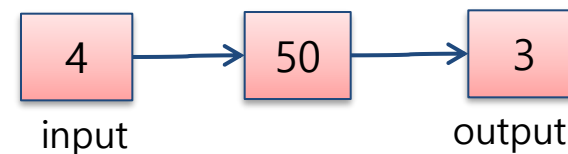
Sequential 클래스 이용

1절. 텐서플로우 케라스 > 1.1. 인공신경망 모델을 만드는 방법



예

```
1. from tensorflow.keras import Sequential
2. from tensorflow.keras.layers import Input, Dense
3.
4. model = Sequential()
5. model.add(Input(shape=4))
6. model.add(Dense(50, activation='relu'))
7. model.add(Dense(3, activation='softmax'))
8. model.summary()
```



Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 50)	250
dense_1 (Dense)	(None, 3)	153

=====
Total params: 403 (1.57 KB)
Trainable params: 403 (1.57 KB)
Non-trainable params: 0 (0.00 Byte)



장점

- Sequential API로 모델을 설계하는 것은 직관적이고 편리함



단점

- 단순히 층을 쌓는 것만으로는 모든 인공신경망을 구현할 수 없음. 즉, 복잡한 인공 신경망을 구현할 수 없음.

함수형 API 이용

1절. 텐서플로우 케라스 > 1.1. 인공신경망 모델을 만드는 방법

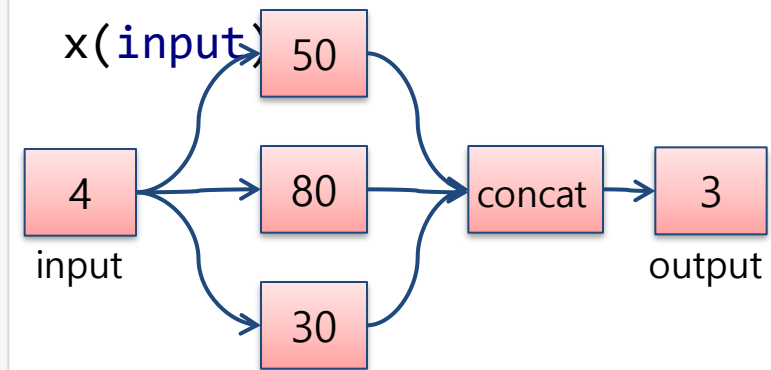
✓ 함수형 API를 이용하면 다중 입력 및 다중 출력 모델 생성 가능

1. `from tensorflow.keras import Model`
2. `from tensorflow.keras.layers import Input, Dense`
3. `from tensorflow.keras.layers import concatenate, Activation`
- 4.
5. `input = Input(shape=(4,))`
6. `dense1 = Dense(50, activation='relu')(input)`
7. `dense2 = Dense(80, activation='relu')(input)`
8. `dense3 = Dense(30, activation='relu')(input)`
9. `x = concatenate([dense1, dense2, dense3])`
10. `output = Dense(3, activation='softmax')(x)`
11. `model = Model(inputs=input, outputs=output)`
12. `model.summary()`

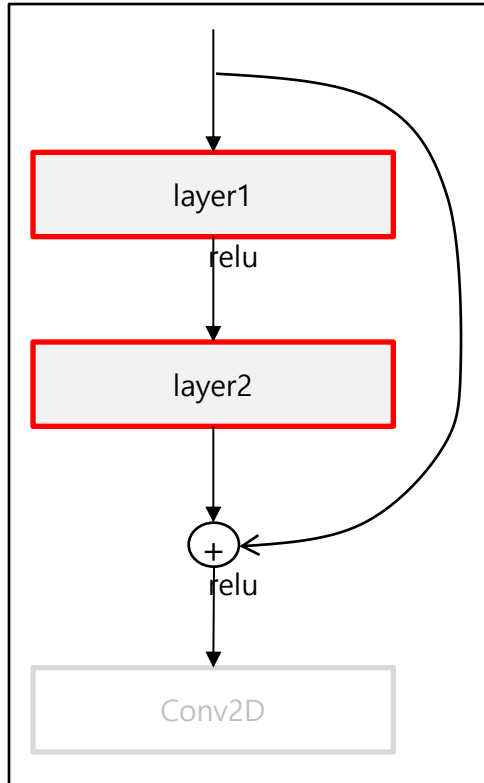
`x = Dense(64)(input)`

위의 코드는 아래의 코드와
같음

`x = Dense(64)`



Residual block 구현



4장. 합성곱 신경망

인공신경망 딥러닝 알고리즘 구현과 실시간 객체 탐지

1절. 합성곱 신경망

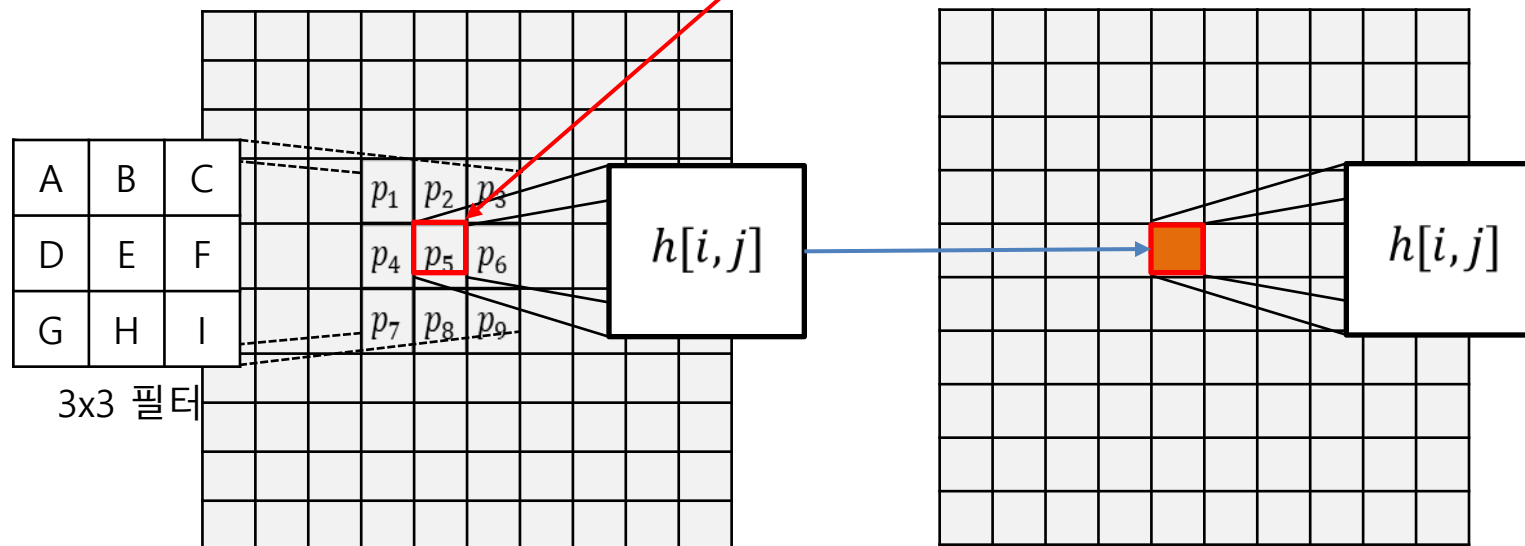


이미지 합성곱

1절. 합성곱 신경망 > 1.2. 이미지 합성곱

이미지는 2차원 이므로 적용하는 필터도 2차원

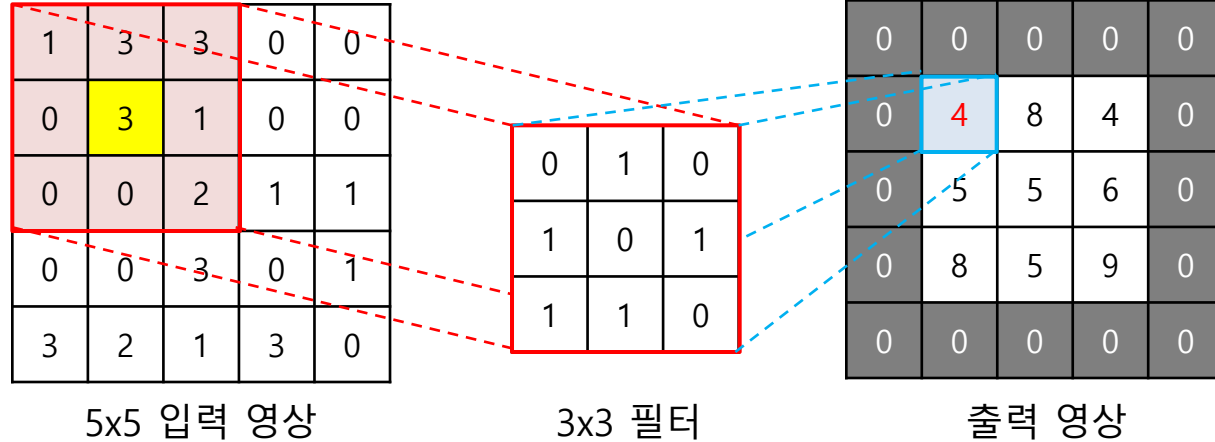
필터를 적용할 수 있는 영역에 대해서 현재 주목화소와 주위 화소의 배열을 각 요소끼리 모두 곱한 후 모든 항목을 더함



$$h[i, j] = A \times p_1 + B \times p_2 + C \times p_3 + D \times p_4 + E \times p_5 + F \times p_6 + G \times p_7 + H \times p_8 + I \times p_9$$

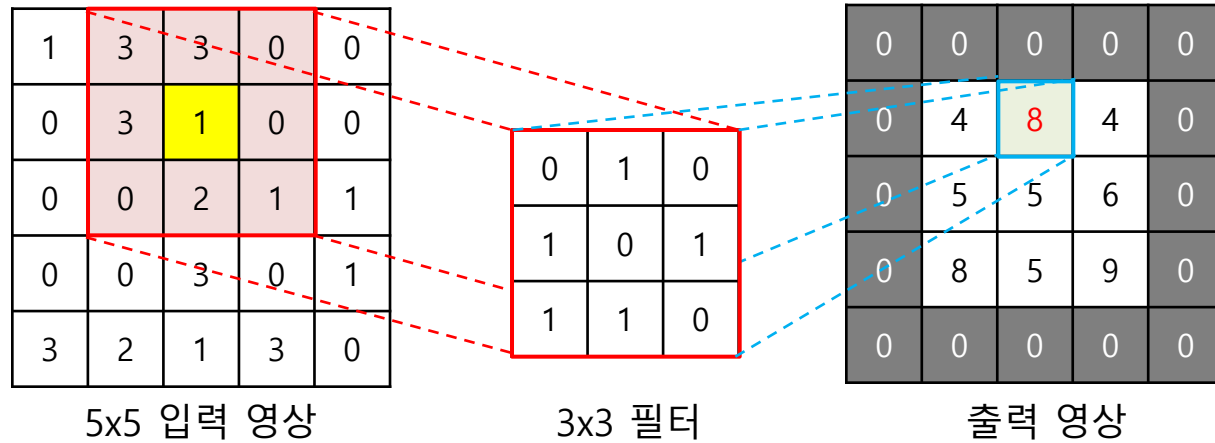
필터 적용

1절. 합성곱 신경망 > 1.2. 이미지 합성곱



Zero padding : 5x5이미지
의 바깥 화소는 필터를 적
용할 수 없으므로 0으로 채
움

$$\begin{aligned} \text{img}[1,1] &= (1 \times 0) + (3 \times 1) + (3 \times 0) + (0 \times 1) \\ &+ (3 \times 0) + (1 \times 1) + (0 \times 1) + (0 \times 1) + (2 \times 0) \\ &= 4 \end{aligned}$$



$$\begin{aligned} \text{img}[1,2] &= (3 \times 0) + (3 \times 1) + (0 \times 0) + (3 \times 1) \\ &+ (1 \times 0) + (0 \times 1) + (0 \times 1) + (2 \times 1) + (1 \times 0) \\ &= 8 \end{aligned}$$

이미지 필터 적용 예

1절. 합성곱 신경망 > 1.2. 이미지 합성곱

필터 커널에 따라 원본 이미지로부터 특성이 강조된 이미지를 얻을 수 있음

0	0	0
0	1	0
0	0	0



원 영상

0	-1	0
-1	5	-1
0	-1	0



Sharpen 필터

1	1	1
1	1	1
1	1	1

$\frac{1}{9}$



Box blur

1	2	1
2	4	2
1	2	1

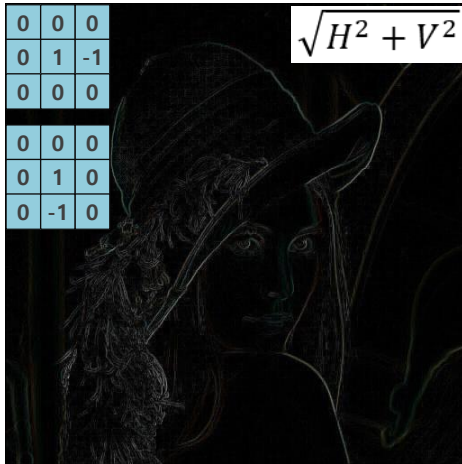
$\frac{1}{16}$



Gaussian blur

0	0	0
0	1	-1
0	0	0
0	0	0
0	1	0
0	-1	0

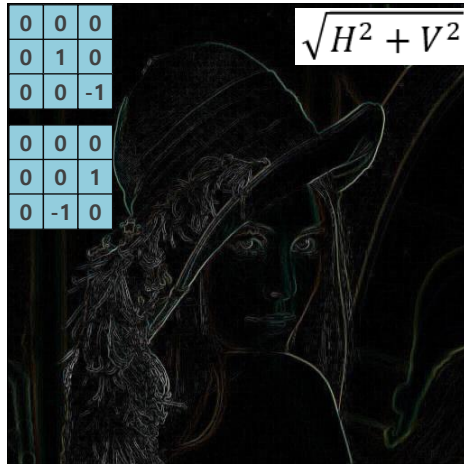
$\sqrt{H^2 + V^2}$



차분 필터

0	0	0
0	1	0
0	0	-1
0	0	0
0	0	1
0	-1	0

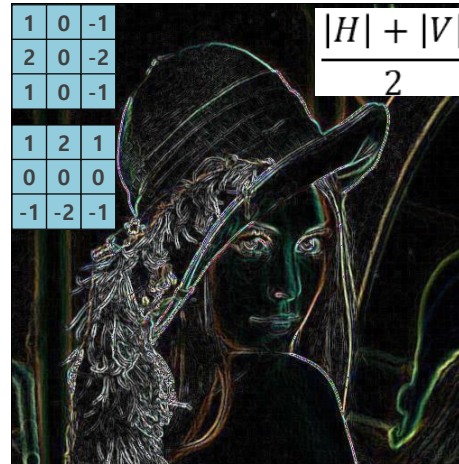
$\sqrt{H^2 + V^2}$



Roberts 필터

1	0	-1
2	0	-2
1	0	-1
1	2	1
0	0	0
-1	-2	-1

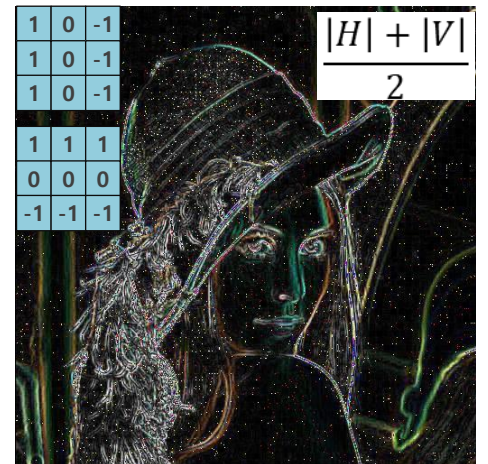
$\frac{|H| + |V|}{2}$



Sobel 필터

1	0	-1
1	0	-1
1	0	-1
1	1	1
0	0	0
-1	-1	-1

$\frac{|H| + |V|}{2}$



Prewitt 필터

1절. 합성곱 신경망 > 1.2. 이미지 합성곱

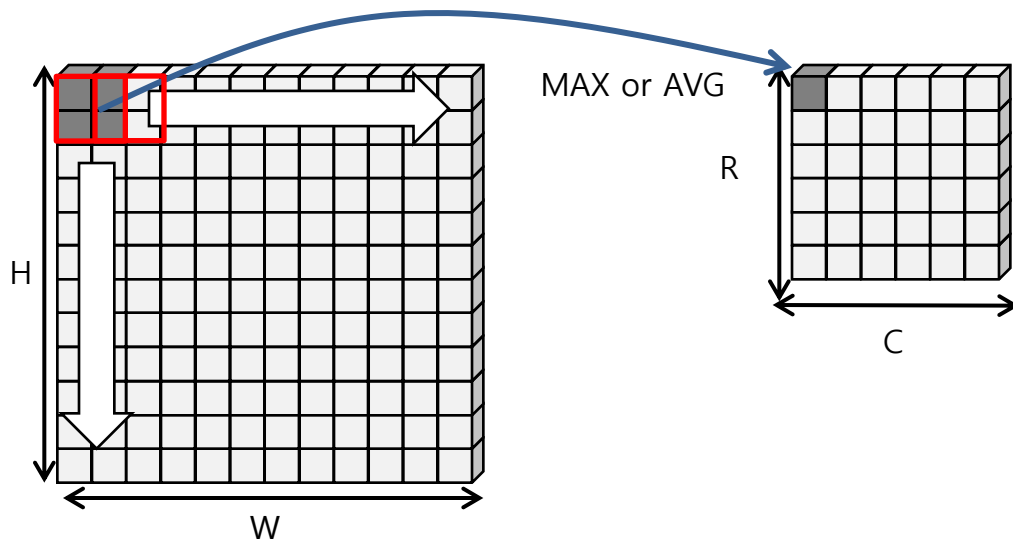
[illegible]

현재 화소와 현재 화소 주위 화소들을 이용해 필터링 처리 해야 하므로 이미지의 $[1,1]$ 화소부터 $[27,27]$ 화소까지만 필터링 처리 할 수 있다. $[0, 0]$ 화소는 x 좌표, y 좌표가 음수 값이 없다.

풀링

1절. 합성곱 신경망 > 1.3. 풀링

입력 이미지를 차원 축소해서 크기를 줄이는 것

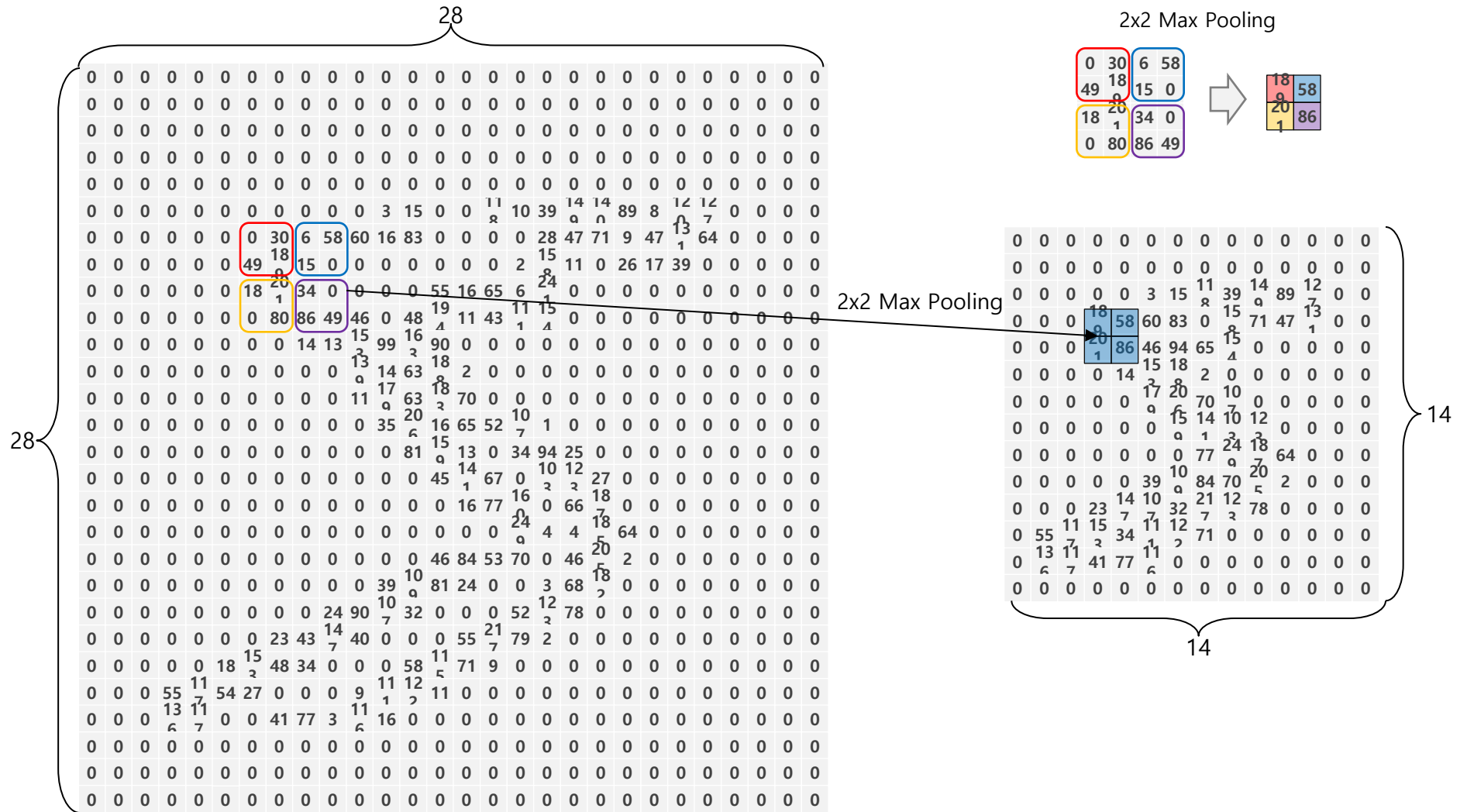


풀링(Pooling) 방법

- Max Pooling : 대상 화소 중에서 가장 큰 화소 값을 선택
- Mean Pooling(Average Pooling) : 대상 화소들의 평균값을 선택
- Min Pooling : 대상 화소 중에서 가장 작은 값을 선택
- Overlapped Pooling : 풀링 사이즈보다 스트라이드가 작을 경우

Max Pooling

1절. 합성곱 신경망 > 1.3. 풀링

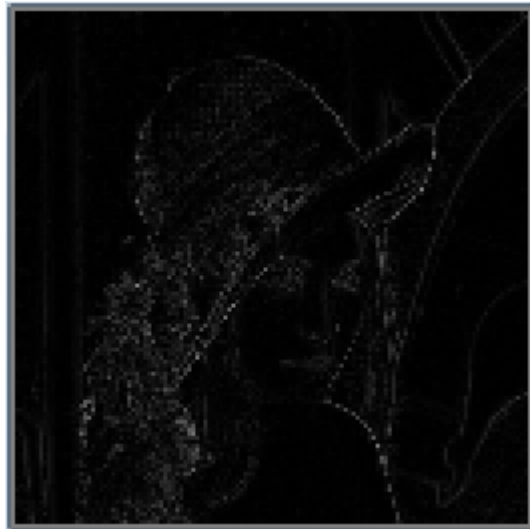


Resize vs. Max pooling

1절. 합성곱 신경망 > 1.3. 풀링



Resize 1/2



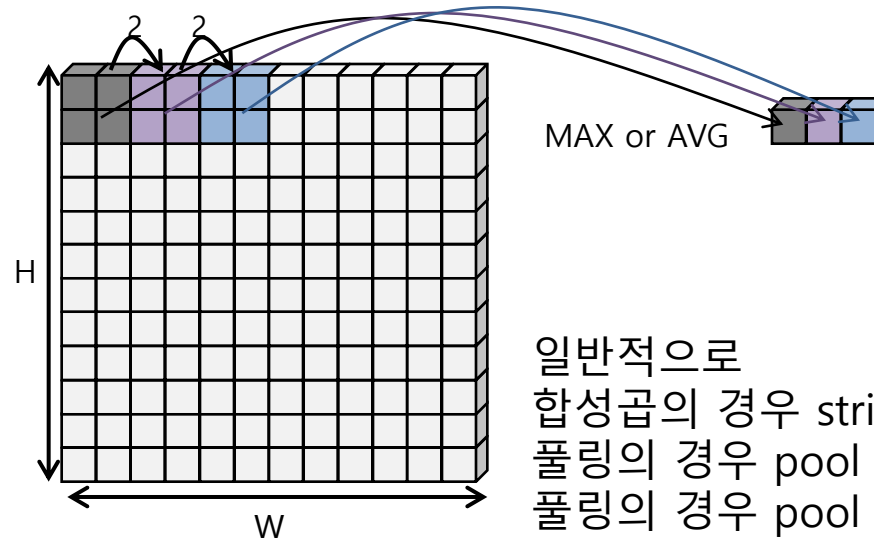
2x2 Max pooling



스트라이드

1절. 합성곱 신경망 > 1.4. 스트라이드와 제로 패딩

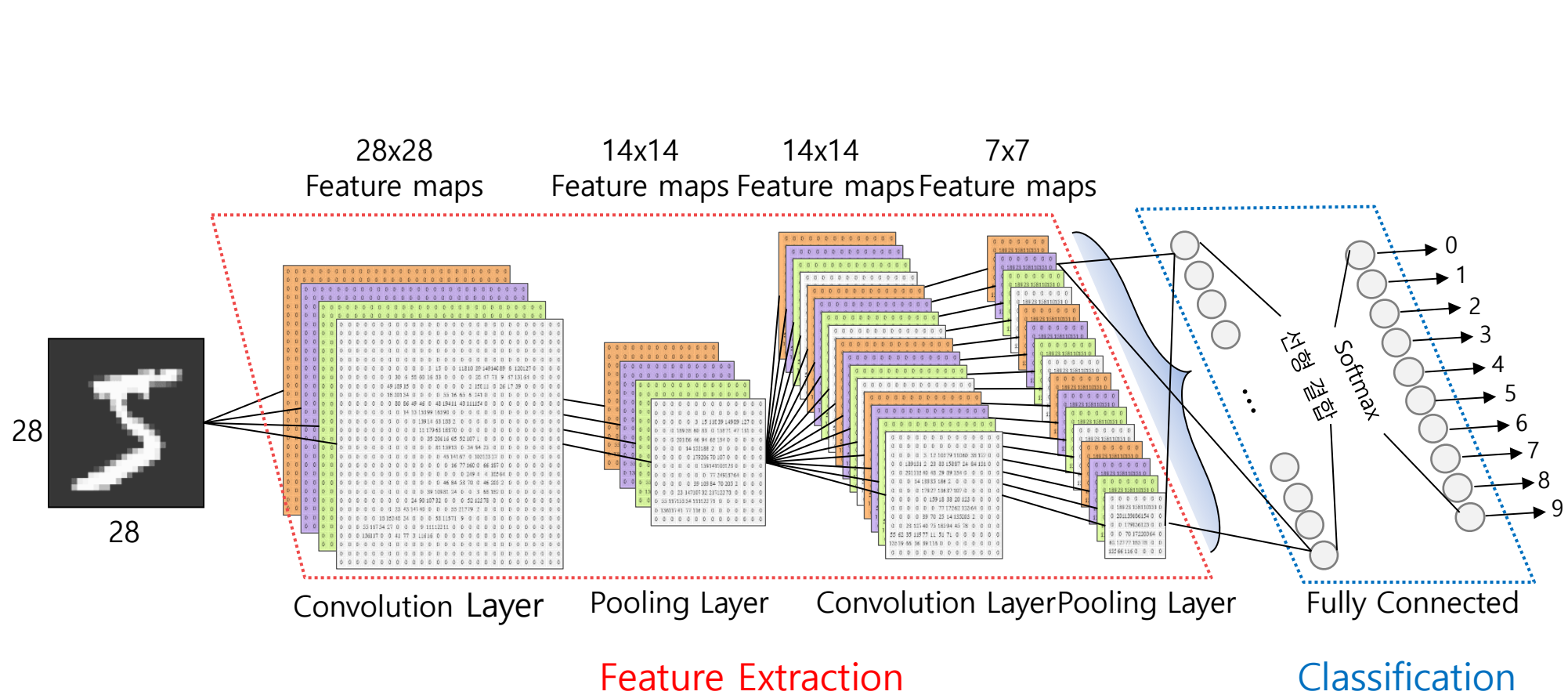
스트라이드 : 합성곱 또는 풀링 과정에서 필터가 이미지에 적용된 후 움직이는 크기



일반적으로
합성곱의 경우 $\text{strides}=1$
풀링의 경우 pool size 와 같음
풀링의 경우 pool size 보다 작으면 overlapped pooling이라고 함

CNN 모델

1절. 합성곱 신경망 > 1.5. CNN 모델



2절. 손글씨 숫자 인식을 위한 CNN 구현

4장. 합성곱 신경망



손글씨 숫자 인식하기

2절. 손글씨 숫자 인식을 위한 CNN 구현 > 2.1. 손글씨 숫자 인식하기

- 우편번호 자동 분류기처럼 숫자를 분류할 수 있도록 만들기 위해 필요한 것
 - 딥러닝 구현 알고리즘
 - 손으로 쓴 숫자 데이터
- 알고리즘은?
 - 인공신경망 CNN 사용
- 데이터는?
 - 종이에 0~9까지 숫자를 쓴다. → 대략 1000개 이상은 써야 하지 않을까?
 - 숫자를 이미지로 저장한다.
 - 개별 숫자를 잘라내어 파일로 저장한다.
 - 전처리 한다.
 - 숫자의 크기를 일정한 크기로 자르고, 밝기 값 조정 등...
 - 레이블을 지정한다.

MNIST 손글씨 숫자 데이터

2절. 손글씨 숫자 인식을 위한 CNN 구현 > 2.2. MNIST 손글씨 숫자 데이터

- MNIST(Modified National Institute of Standards and Technology)
- 미국의 NIST에서 이미지 처리 시스템을 위해 모은 0부터 9까지의 숫자 이미지 데이터셋
- 각 이미지는 28x28 크기, 이것을 펼치면 784 차원의 벡터

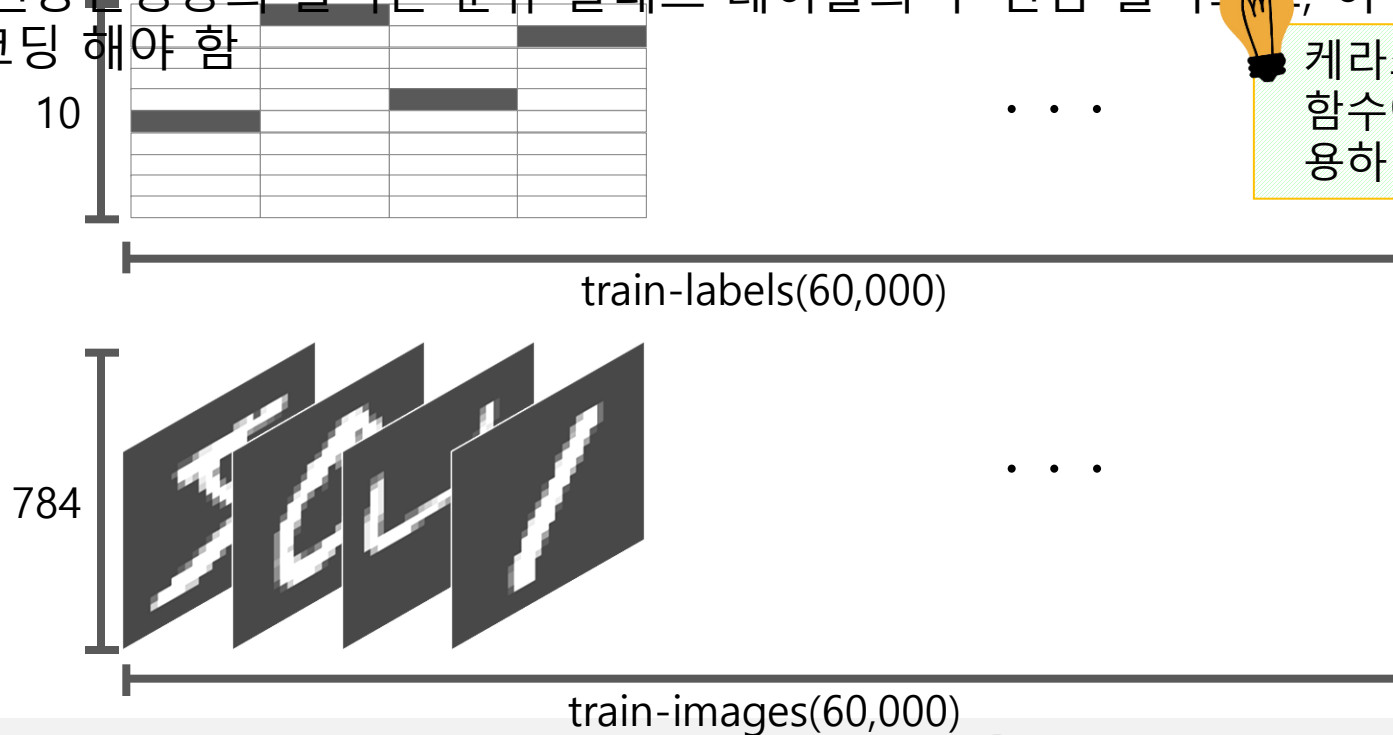


28 x 28 pixels image 이미지를 784개 화소(Pixel)의 숫자로 표현

인공신경망에서 원-핫 인코딩

2절. 손글씨 숫자 인식을 위한 CNN 구현 > 2.2. MNIST 손글씨 숫자 데이터

- 원-핫 인코딩(One-Hot Encoding)은 범주형 데이터(카테고리 데이터)를 수치 데이터로 변환하는 데 사용되는 기술
- 원-핫 인코딩의 핵심 아이디어는 각 카테고리 값을 이진 벡터로 표현하는 것
 - 각 카테고리는 해당하는 인덱스 위치의 원소만 1이고 나머지 원소는 모두 0으로 표현됨
 - 이렇게 하면 모든 카테고리가 수치 데이터로 변환되며, 각 카테고리 간에 상호간섭이 없게 됨
- 인공신경망은 멀티클래스 분류일 경우 출력층의 뉴런의 개수가 클래스 라벨의 수와 일치해야 하는데, 그러면 인공신경망의 출력은 분류 클래스 레이블의 수 만큼 출력되며, 이 값이 정답과 비교되려면 정답을 원-핫 인코딩 해야 함



케라스에서는 멀티클래스 분류를 위한 손실 함수에 `sparse_categorical_crossentropy`를 사용하면 원-핫 인코딩 하지 않아도 됩니다.

합성곱 층을 위한 클래스 Conv2D

2절. 손글씨 숫자 인식을 위한 CNN 구현 > 2.3. 손글씨 숫자 인식을 위한 케라스 CNN 모델



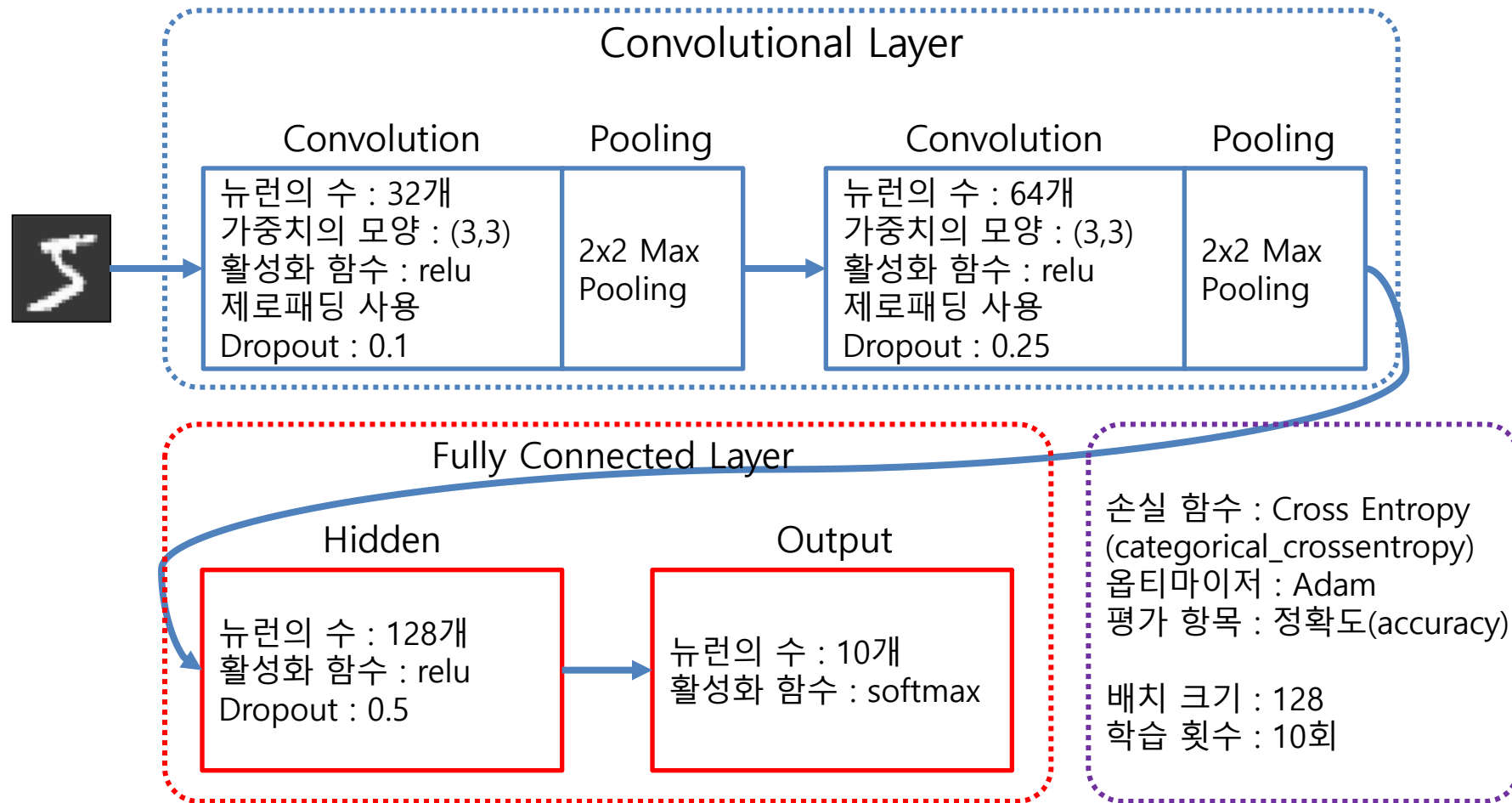
```
tf.keras.layers.Conv2D(
```

```
    filters, kernel_size,  
    strides=(1, 1), padding='valid',  
    data_format=None, dilation_rate=(1, 1),  
    activation=None, use_bias=True,  
    kernel_initializer='glorot_uniform',  
    bias_initializer='zeros',  
    kernel_regularizer=None, bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None, bias_constraint=None
```

```
)
```

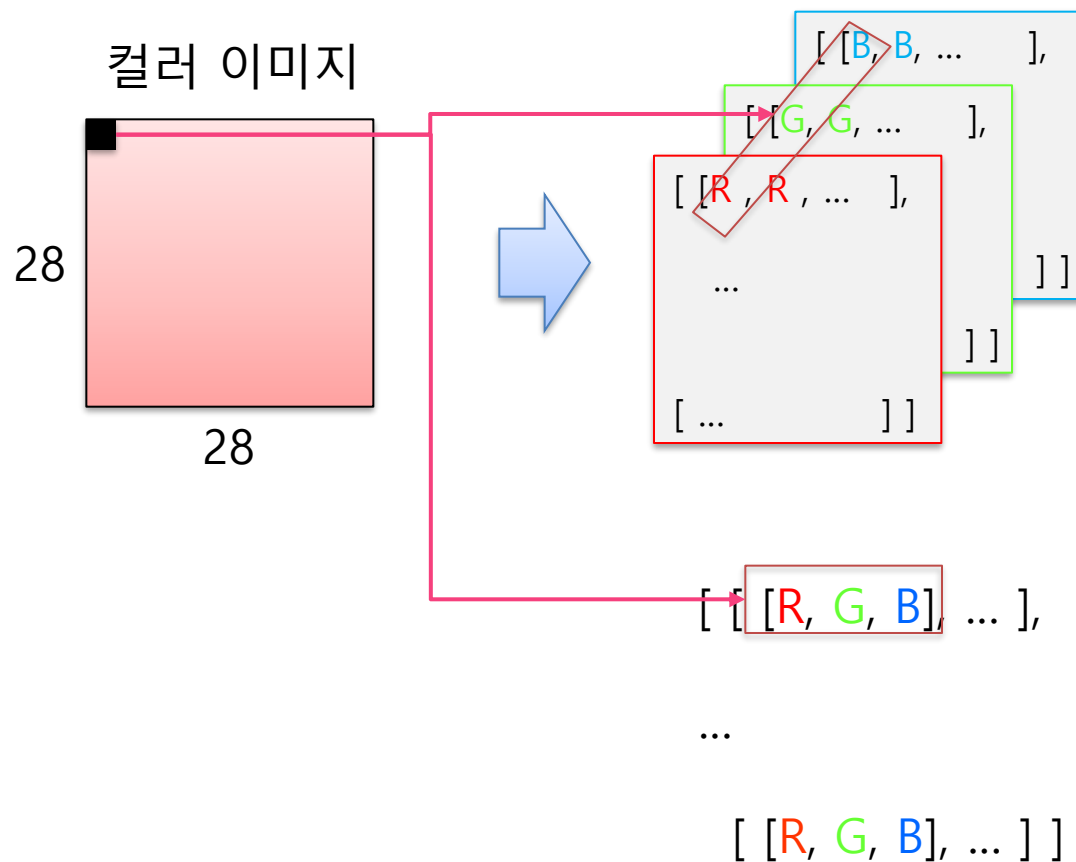

손글씨 숫자 인식을 위한 케라스 CNN 모델

2절. 손글씨 숫자 인식을 위한 CNN 구현 > 2.3. 손글씨 숫자 인식을 위한 케라스 CNN 모델



Channel First와 Channel Last

2절. 손글씨 숫자 인식을 위한 CNN 구현



CNN 코드 (1/2) – 모델 정의하기

2절. 손글씨 숫자 인식을 위한 CNN 구현 > 2.3. 손글씨 숫자 인식을 위한 케라스 CNN 모델



모델 정의

```
1. from tensorflow.keras import Sequential, layers
2. model = Sequential([
3.     layers.Input(shape=(28,28,1)),
4.     layers.Conv2D(32, (3,3), activation='relu'),
5.     layers.MaxPooling2D(), # pool_size=(2,2)가 기본값
6.     layers.Dropout(0.1),
7.     layers.Conv2D(64, 3, activation='relu'),
8.     layers.MaxPooling2D(),
9.     layers.Dropout(0.25),
10.    layers.Flatten(),
11.    layers.Dense(128, activation='relu'),
12.    layers.Dropout(0.5),
13.    layers.Dense(10, activation='softmax')
14. ])
15. model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 24, 24, 32)	832
max_pooling2d (MaxPooling2D)	(None, 12, 12, 32)	0
dropout (Dropout)	(None, 12, 12, 32)	0
conv2d_1 (Conv2D)	(None, 10, 10, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
dropout_1 (Dropout)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204928
dropout_2 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1290

=====
Total params: 225546 (881.04 KB)
Trainable params: 225546 (881.04 KB)
Non-trainable params: 0 (0.00 Byte)
=====

CNN 코드 (2/2) - 데이터 불러오고 학습한 후 평가하기

2절. 손글씨 숫자 인식을 위한 CNN 구현 > 2.3. 손글씨 숫자 인식을 위한 케라스 CNN 모델

✓ 훈련 정의

```
1. model.compile(loss='sparse_categorical_crossentropy',  
2.               optimizer='adam', metrics=['accuracy'])
```

✓ 데이터 불러오기

```
1. from tensorflow.keras.datasets import mnist  
2. (X_train, y_train), (X_test, y_test) = mnist.load_data()  
3. X_train = X_train.reshape(-1, 28, 28, 1) / 255.0  
4. X_test = X_test.reshape(-1, 28, 28, 1) / 255.0
```

✓ 데이터 학습

```
1. model.fit(X_train, y_train, batch_size=128, epochs=10, verbose=1)
```

✓ 모델 평가

```
1. loss, accuracy = model.evaluate(test_X, test_y, verbose=0)  
2. print(f'Test loss: {loss}, Test accuracy: {accuracy}')
```